Reviewer Pack — Beck-Fiala Slack of Clause-Variable Hypergraph Bounds DPLL L...

Entry dbaf5e8ccbe1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 23:24:45 UTC

Beck-Fiala Slack of Clause-Variable Hypergraph Bounds DPLL Leaves

Entry ID: `dbaf5e8ccbe1` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-27 23:24:45 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial discrepancy theory: the Beck-Fiala slack delta(H) = max_S |sum_{e in S} chi(e)| - 2t+1 of a t-uniform set system H, computed via the Beck-Fiala floating-color rounding algorithm (iteratively freeze tight constraints; rarely deployed against proof complexity) **Field B** (complexity object): DPLL search-tree leaf count L_DPLL(F) of small unsatisfiable 3-CNF formulas under a fixed lex variable order with unit propagation, equivalently tree-resolution leaf count

Statement:

> Let F be an unsatisfiable 3-CNF on n<=20 variables with m clauses, and let H_F be the 3-uniform hypergraph whose vertices are the m clauses and whose hyperedges are the variable-stars E_v = {C : v in C}; let delta(F) be the Beck-Fiala slack of H_F obtained by Beck-Fiala floating-color rounding (max over all variable subsets S of |sum_{v in S} chi(E_v)| minus the trivial 2t-1=5 bound, where chi:clauses->{-1,+1} is the discrepancy coloring). Then for every unsatisfiable 3-CNF F at clause density alpha in [4.0,5.5], log2(L_DPLL(F)) >= delta(F)/3 - 1. A single F with delta(F)/3 - 1 > log2(L_DPLL(F)) + 0.5 falsifies the conjecture.

Rationale (proposer's reasoning):

> Beck-Fiala discrepancy of the variable-star hypergraph quantifies the unavoidable imbalance any +/-1 coloring of clauses must accept, which intuitively obstructs tree-resolution from finding a small witnessing subtree because each branch must resolve a near-balanced set of clauses; this connects classical Beck-Fiala slack (a Chazelle/Matousek-style dispersion invariant) to DPLL leaf complexity through clause coloring rather than variable coloring. Unlike subdeterminant or Kashin-split approaches that target rigidity, this targets proof complexity directly via a hypergraph attached canonically to F. The mapping uses only Beck-Fiala floating-color rounding (~30 lines of pure Python) and standard lex-DPLL, sidestepping advanced algebraic machinery and natural-proofs concerns since delta(F) is not a property of a Boolean function class but of a specific CNF.

Taxonomy category: `DISPERSION_DISCREPANCY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `36859e1f258b7cca`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across 200 unsatisfiable 3-CNFs (n in {12,14,16,18,20}, alpha in {4.0,4.5,5.0,5.5}), define gap_i = log2(L_DPLL(F_i)) - delta(F_i)/3 + 1. SUPPORT requires gap_i >= 0 on >=95% of instances AND min_i gap_i >= -0.5. FALSIFIED if any instance has gap_i < -0.5.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.92	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Beck-Fiala discrepancy 3-CNF DPLL tree resolution lower bound` - `combinatorial discrepancy hypergraph coloring proof complexity unsatisfiable CNF` - `set system discrepancy floating color rounding tree-like resolution size`

Top relevant hits considered: - [http://arxiv.org/abs/2602.09948v1] Non-Additive Discrepancy: Coverage Functions in a Beck-Fiala Setting - [http://arxiv.org/abs/2508.01937v1] An Improved Bound for the Beck-Fiala Conjecture - [http://arxiv.org/abs/1306.6081v3] An improvement of the Beck-Fiala theorem - [http://arxiv.org/abs/0905.1587v5] Unsatisfiable Linear CNF Formulas Are Large and Complex - [http://arxiv.org/abs/0806.1148v2] Unsatisfiable CNF Formulas need many Conflicts - [http://arxiv.org/abs/1908.06789v2] Combinatorial Proof of the Minimal Excludant Theorem - [http://arxiv.org/abs/2309.09631v1] Digital analysis of early color photographs taken using regular color screen processes - [http://arxiv.org/abs/1004.3374v1] On the precision attainable with various floating-point number systems

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
import json

def generate_3cnf(n, m):
    variables = list(range(1, n + 1))
    clauses = []
    while len(clauses) < m:
```

```
        clause = set(random.sample(variables, 3))
        if all(len(set(c).intersection(F)) == 0 for F in clauses):
            clauses.append(tuple(sorted(clause)))
    return clauses

def lex_dpll(F, assignment=None):
    if assignment is None:
        assignment = {}
    if len(assignment) == len(F[0]):
        return True
    var = next(v for v in range(1, len(F[0]) + 1) if v not in assignment)
    for val in [-1, 1]:
        assignment[var] = val
        if lex_dpll(F, assignment):
            return True
        del assignment[var]
    return False

def beck_fiala_slack(F):
    n = len(F[0])
    m = len(F)
    H_F = [[] for _ in range(m)]
    for i, clause in enumerate(F):
        for var in clause:
            H_F[i].append(var)

    def lp_solve(A, b):
        A_t = list(zip(*A))
        pseudo_inv = [[sum(a * b for a, b in zip(row, col)) / sum(a**2 for a in row) if any(b != 0
        return [sum(pseudo_inv[i][j] * b[j] for j in range(len(b))) for i in range(len(A))]

    def max_discrepancy(chi):
        max_disc = 0
        for S in range(1 << n):
            disc = sum(chi[i] if (S >> i) & 1 else -chi[i] for i in range(n))
            if abs(disc) > max_disc:
                max_disc = abs(disc)
        return max_disc

    chi = [0] * m
    while True:
        A = []
        b = []
        for i, clause in enumerate(F):
            A.append([1 if var in clause else 0 for var in range(n)])
            b.append(0)
        slack = lp_solve(A, b)
        max_slack = max(slack)
        if max_slack <= 2:
            break
        for i, s in enumerate(slack):
            if abs(s) == max_slack:
                chi[i] = -1 if random.random() < 0.5 else 1
                break

    return max_discrepancy(chi)
```

```python
def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [12, 14, 16, 18, 20]
    alpha_values = [4.0, 4.5, 5.0, 5.5]
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        m = int(n * alpha)
        F = generate_3cnf(n, m)
        if not lex_dpll(F):
            continue
        delta_F = beck_fiala_slack(F)
        L_DPLL_F = 0
        while True:
            assignment = {}
            if lex_dpll(F, assignment):
                L_DPLL_F += 1
            else:
                break
        instances_tested += 1
        gap = math.log2(L_DPLL_F) - delta_F / 3 + 1
        if gap < -0.5:
            conjecture_holds = False
            counterexample = f"n={n}, alpha={alpha}, L_DPLL(F)={L_DPLL_F}, delta(F)={delta_F}"
            break

    return {
        "metric_name": "gap",
        "metric_value": gap,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    mean_gap = sum(r["metric_value"] for r in results) / len(results)
    std_gap = math.sqrt(sum((r["metric_value"] - mean_gap) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["gap"] >= 0 for r in results):
        print(f"RESULT: SUPPORTED mean={mean_gap} std={std_gap} support_fraction={support_fractior
    elif any(r["gap"] < -0.5 for r in results):
        first_failing_seed = next(r["seed"] for r in results if r["gap"] < -0.5)
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
    else:
        print("RESULT: INCONCLUSIVE")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41bdba6b.py", line 119, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41bdba6b.py", line 88, in run_trial
    m = int(n * alpha)
                ^^^^^
NameError: name 'alpha' is not defined
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed with a NameError (‘alpha’ is not defined) before producing any data, so no instances were evaluated against the pre-registered criterion. | next: Fix the scoping bug in run_trial so that alpha is passed in or defined in scope, then rerun the 200-instance sweep.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	21786
2	preregistration	claude_max	opus	0	0	5349
3	novelty	claude_max	opus	0	0	3881
4	novelty	claude_max	opus	0	0	7154
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16397
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13361
7	judge	claude_max	opus	0	0	3368

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 71296 ms total latency. Provider mix: {‘claude_max’: 5, ‘ollama_remote’: 2}

(full prompt+response transcripts available in `research/audit/dbaf5e8ccbe1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/dbaf5e8ccbe1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/dbaf5e8ccbe1.tar.gz` (if generated)